



Spring 2026

MCT344 / MCT342 Industrial Robotics

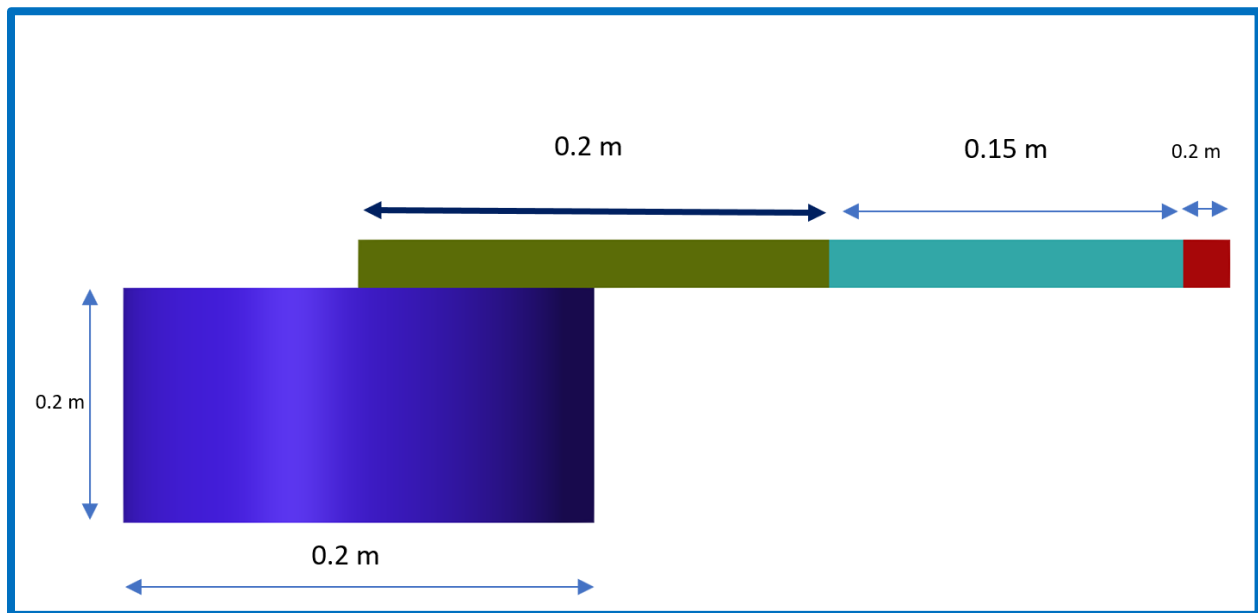
Lab Sheet 1

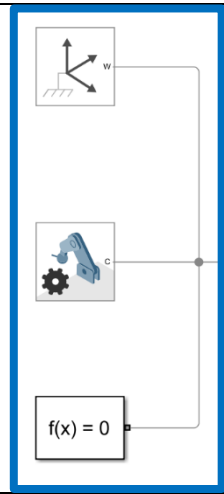
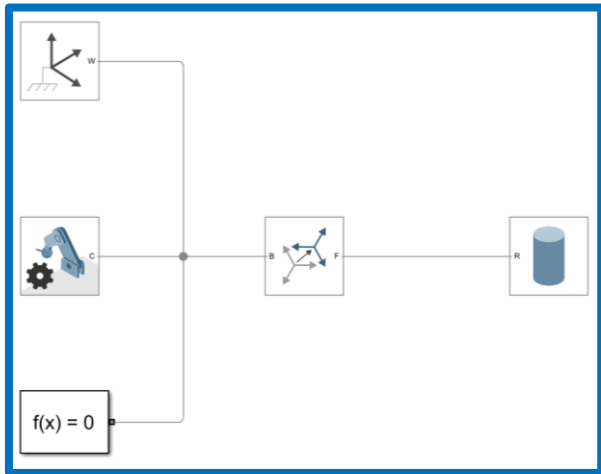
MATLAB & Simulink

Lab1: Robot Creation and Forward Kinematics

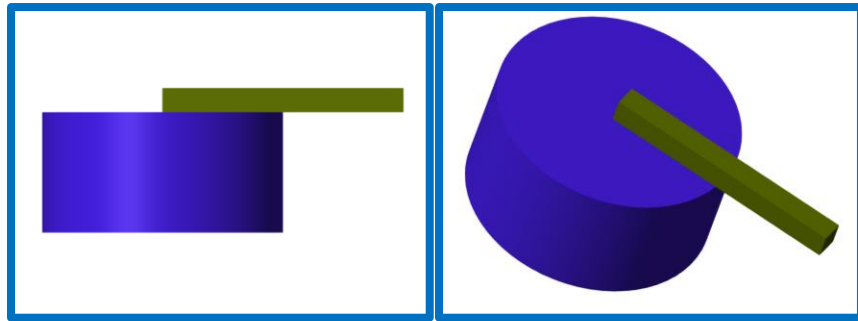
In this lab, you are required to create the following RR Robot with these dimensions using Simscape tool box, and apply forward kinematics to it.

Part 1: Using Simscape Toolbox to Create Robot



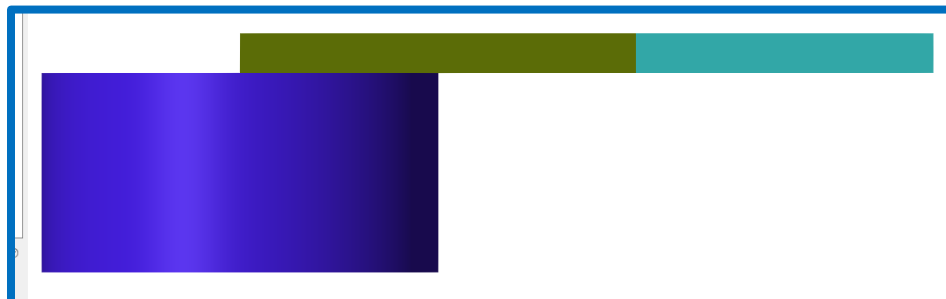
Step	Action	Screenshot
1	Simscape > Utilities > Solver Configuration Simscape > Multibody > Frames and Transforms > World Frame Simscape > Multibody > Utilities > Mechanism Configuration	
2	A Simscape > Multibody > Body Element > Cylindrical Solid Cylindrical Solid Block: Modify length, radius and color Note: By default, the frame of any rigid body we create is at its center	
	B Simscape > Multibody > Frames and Transforms > Rigid Transform Rigid Transform Block: B === World frame F === Cylindrical Solid Block Translation > Method = Cartesian Modify offset to match half of the height of the cylinder, so that the base of the cylinder is setting on the world frame	

When you run, you should have something like this:



Step	Action	Screenshot
4	A Copy These 4 Blocks	
	B <u>Brick Block:</u> Modify length and color only	
	C <u>Rigid Transform Block:</u> (Between Revolute Joint and link 1 (Brick)) Modify offset to match half of the length of the link.	
	D <u>Rigid Transform Block:</u> (Between Revolute Joint and Brick 2) Modify offset to match half of the length of the link 2	

Currently, when you run, you should have something like this:



Step	Action	Screenshot
6	A	Group Each Link in a Subsystem for a more clear Visual using the 3 dots when you select blocks
	B	We want to reach this simplified version:

When you run, you should have something like this:



Part 2: Using Robotic Systems Toolbox to Apply Forward Kinematics

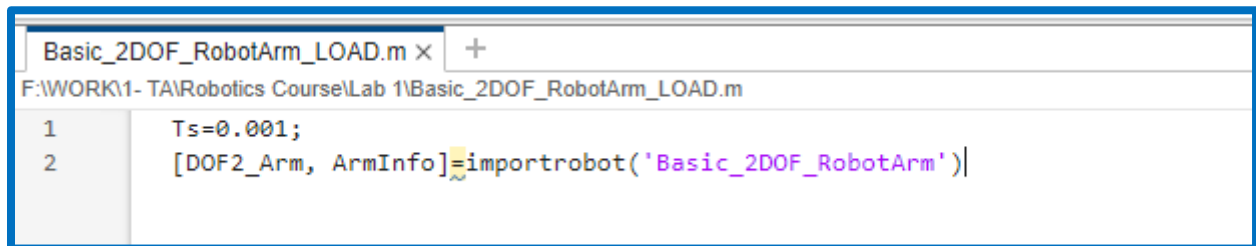
First, we need to create a Rigid Body Tree representation for our robot.

To do this we will use MATLAB:

Creating a MATLAB (.m) file

MATLAB Home Tab > New > Script

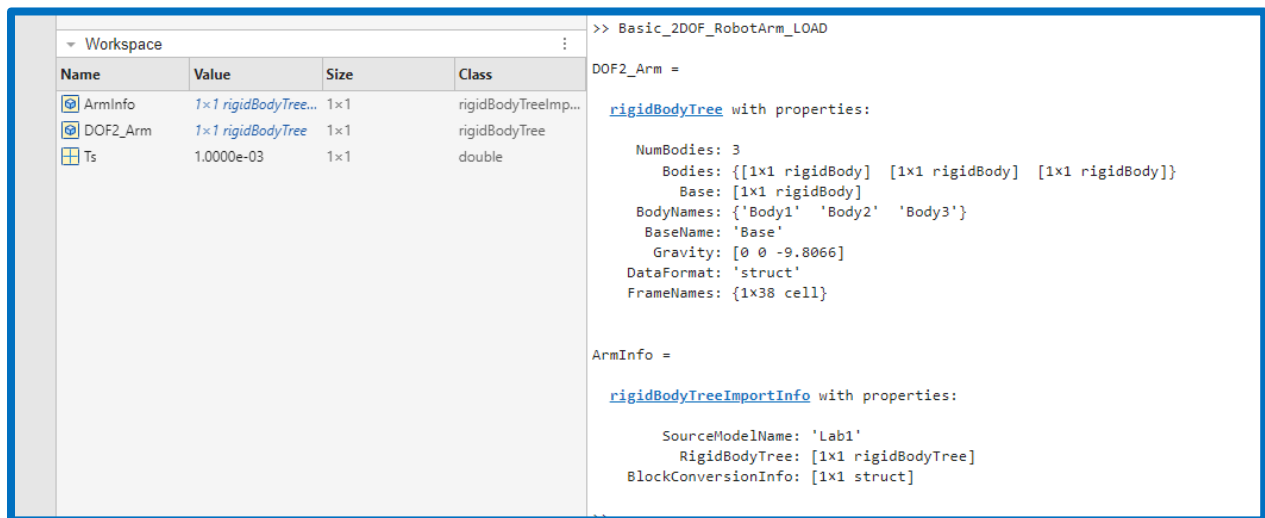
```
Ts=0.001;  
[DOF2_Arm, ArmInfo]=importrobot(** YOUR Simulink model file NAME **)
```



The screenshot shows a MATLAB script editor window titled 'Basic_2DOF_RobotArm_LOAD.m'. The file path is 'F:\WORK1- TA\Robotics Course\Lab 1\Basic_2DOF_RobotArm_LOAD.m'. The script contains two lines of code:

```
1 Ts=0.001;  
2 [DOF2_Arm, ArmInfo]=importrobot('Basic_2DOF_RobotArm')
```

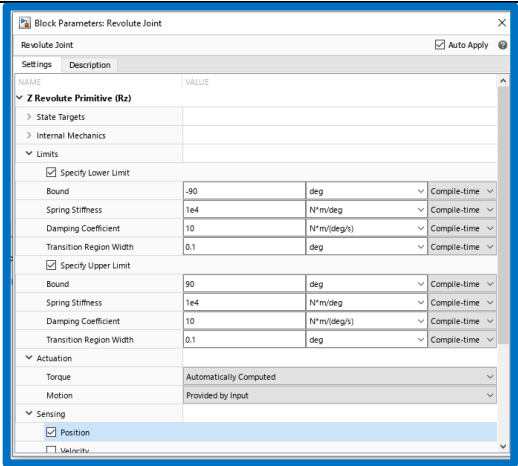
This should appear:

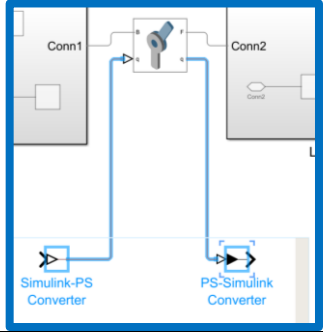
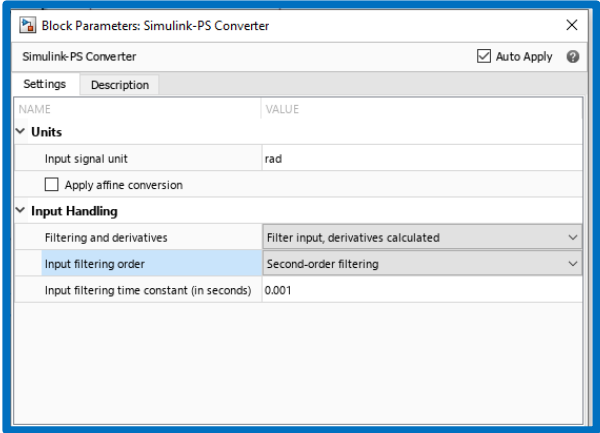


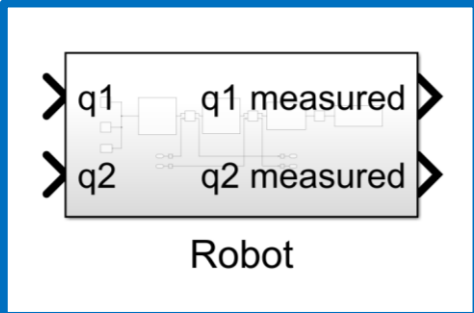
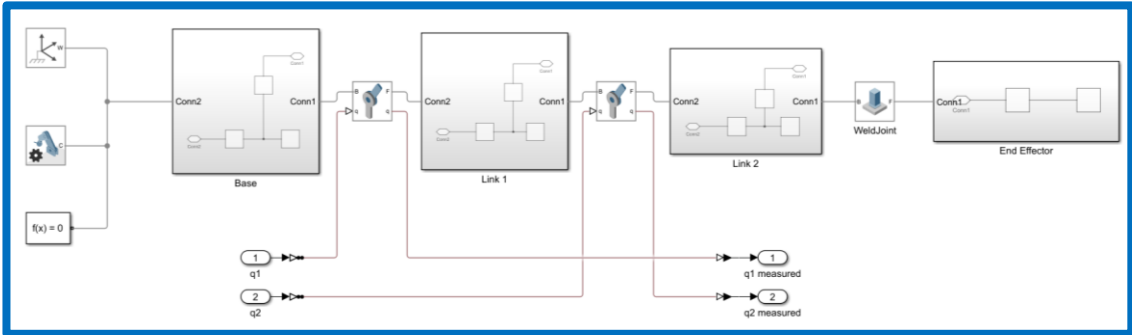
The screenshot shows the MATLAB workspace and command window. The workspace contains three variables: ArmInfo (1x1 rigidBodyTreeImportInfo), DOF2_Arm (1x1 rigidBodyTree), and Ts (1x1 double). The command window shows the output of the importrobot function:

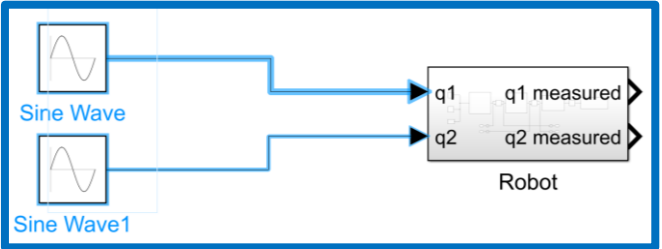
```
>> Basic_2DOF_RobotArm_LOAD  
DOF2_Arm =  
    rigidBodyTree with properties:  
        NumBodies: 3  
        Bodies: {[1x1 rigidBody] [1x1 rigidBody] [1x1 rigidBody]}  
        Base: [1x1 rigidBody]  
        BodyNames: {'Body1' 'Body2' 'Body3'}  
        BaseName: 'Base'  
        Gravity: [0 0 -9.8066]  
        DataFormat: 'struct'  
        FrameNames: {1x38 cell}  
  
ArmInfo =  
    rigidBodyTreeImportInfo with properties:  
        SourceModelName: 'Lab1'  
        RigidBodyTree: [1x1 rigidBodyTree]  
        BlockConversionInfo: [1x1 struct]
```

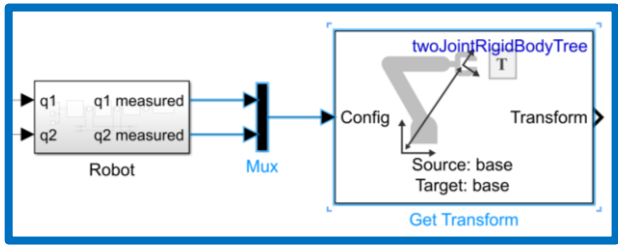
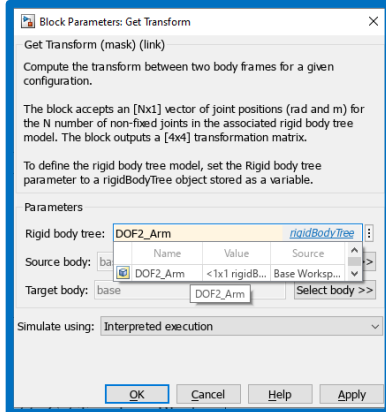
Second, we prepare the joints to be able to take input from us:

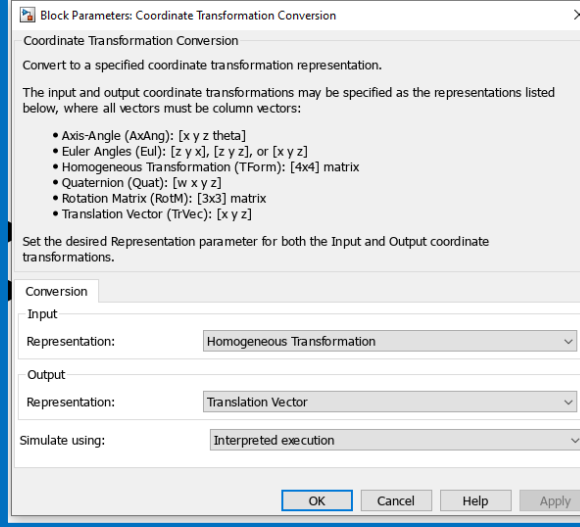
Step	Action	Screenshot
2	A Open the revolute Joint Block	
	B Revolute Joint Block: Limits Tab: Specify Lower Limit > -90 Specify Upper Limit > 90	
	C Revolute Joint Block: Actuation Tab: Motion >> Provided by input Torque >> Automatically Computed	
	D Revolute Joint Block: Sensing Tab: Position >> Check	
	E Then, Repeat the process for the other Revolute Joint.	

Step	Action	Screenshot
3	A Simscape Toolbox >> Utilities >> PS-Simulink Converter Simscape Toolbox >> Utilities >> Simulink-PS Converter	
	B Simulink-PS Converter Block: Units Tab: Input Signal Unit >> Rad	
	C Simulink-PS Converter Block: Input-Handling Tab: Filtering and Derivatives >> Filter input, Derivative calculated Input Filtering Order >> Second Order Filtering Input Filtering Time Constant >> 0.001	
	D Then, Copy the 2 converters, for the other Revolute Joint	

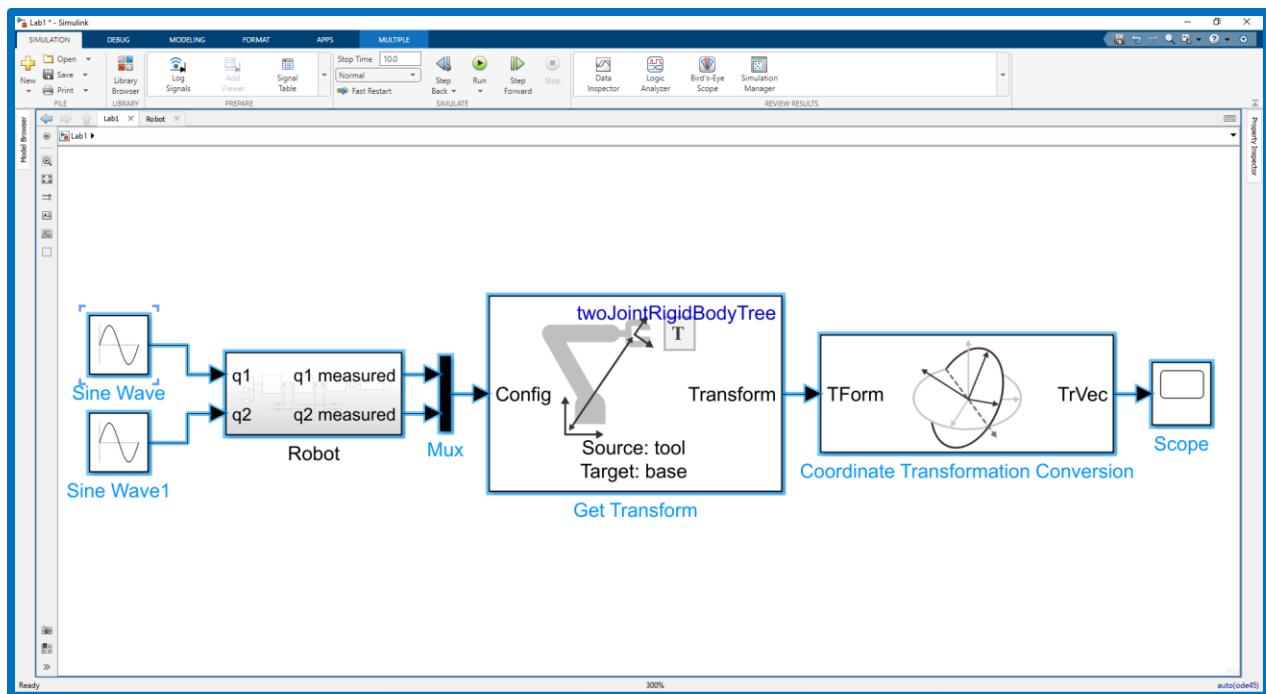
Step	Action	Screenshot						
4	A Select the Whole model now and make it one subsystem called “Robot”							
	B <u>Rename the Inputs and Outputs (the converters Ports) as Follows:</u>							
	<table><tr><td>Rev Joint 1 Input</td><td>q1</td></tr><tr><td>Rev Joint 2 Input</td><td>q2</td></tr><tr><td>Rev Joint 1 Output</td><td>q1 measured</td></tr><tr><td>Rev Joint 2 Output</td><td>q2 measured</td></tr></table>		Rev Joint 1 Input	q1	Rev Joint 2 Input	q2	Rev Joint 1 Output	q1 measured
Rev Joint 1 Input	q1							
Rev Joint 2 Input	q2							
Rev Joint 1 Output	q1 measured							
Rev Joint 2 Output	q2 measured							
C								

Step	Action	Screenshot
5	A Insert Sine wave	
	B <u>Sine wave Block 1:</u> Amplitude >> (Must be less than 90 degrees, but in radians, Less than Pi /2) >> for example: 1.3 Frequency >> for example 1	
	C <u>Sine wave Block 2:</u> Amplitude >> (Must be less than 90 degrees, but in radians, Less than Pi /2) >> for example: 0.6 Frequency >> for example 0.7	
	D Connect both to q1 & q2	

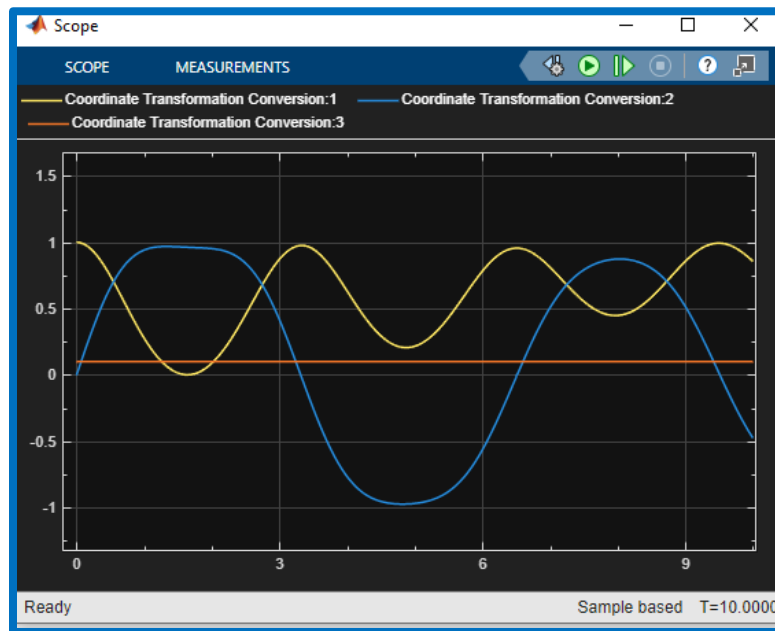
Step	Action	Screenshot
6	A Robotic Systems Toolbox > Manipulator Algorithms > Get Transform <u>Note: Get Transform Needs:</u> 1- Configuration 2- Rigid Body Tree	
	B <u>1- Configuration:</u> Insert >> Mux Combine q1m and q2m using the Mux, then provide it to Get Transform Block	
	C <u>2- Rigid Body Tree</u> <u>Get Transform Block:</u> Rigid Body Tree >> DOF2_Arm <i>(the one we created using MATLAB .m file)</i>	
	D <u>Get Transform Block:</u> Source Body >> Body3 <i>(the LAST body, The End Effector)</i>	
	E <u>Get Transform Block:</u> Target Body >> Base <i>(the FIRST body, The Base)</i>	

Step	Action	Screenshot
7	A Robotic Systems Toolbox > Utilities > Coordinate Transformation Conversion	
	B <u>Coordinate Transformation Conversion Block:</u> Input Representation >> Homogenous Transformation Output Representation >> Translation Vector	
	C Take the output of this block to a scope	

Now Your Model should look like this:



And the Scope should display something like this:



Summary

